Enterprise Integration Project; Technical Report

Virtual Power Plant as a Service (VPPaaS) Platform

Documentation: Source Code, Terraform Scripts, Installation Procedures, and Parametrizations

Group 17 | Afonso Carvalho 116482 | Eduardo Silva 106929

Table of Contents

1. Architecture Overview
2. Technology Stack and Rationale
3. Source Code Documentation
4. Terraform Scripts Documentation
5. Installation Procedures
6. Parametrizations Reference

---

1. Architecture Overview

1.1 Information Flow Design

The VPPaaS platform aggregates Distributed Energy Resources (DER); battery storage (BESS), solar inverters (PV), and EV chargers; coordinating them as a virtual power plant. The architecture separates two distinct layers:

Data plane: Microservices are invoked directly via REST. Each microservice owns a single business entity and implements all logic over that entity. When data from different entities needs to be crossed, calls are made directly via REST.

Event streaming: Apache Kafka is the messaging backbone. The Telemetry MS is the only Kafka consumer in the system. All other microservices are exclusively Kafka producers; they receive data via REST calls, apply their business logic, and publish results to Kafka topics.

1.2 Microservice Responsibilities

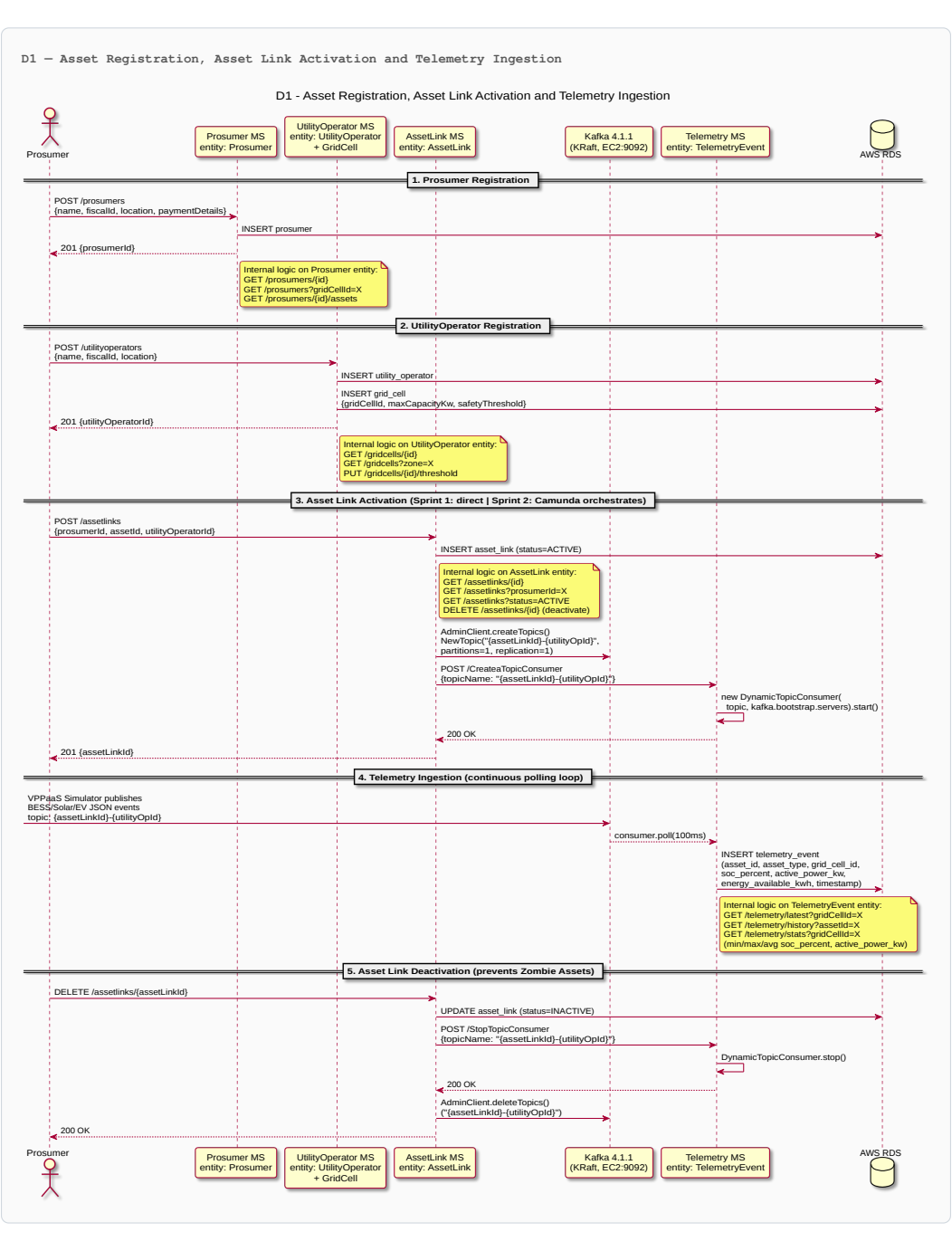
Microservice	Entity Owner	Integration Pattern
Prosumer MS	Prosumer	Pattern 1: REST + JDBC + RDS
UtilityOperator MS	UtilityOperator + GridCell	Pattern 1: REST + JDBC + RDS
AssetLink MS	AssetLink	Pattern 2: REST + JDBC + RDS + Kafka (AdminClient)
Telemetry MS	TelemetryEvent	Pattern 3: Kafka consumer + JDBC + RDS + REST
Flexibility MS	FlexibilityEvent	Pattern 2: REST + JDBC + RDS + Kafka producer
Grid Balancing MS	BalancingRecommendation	Pattern 2: REST + JDBC + RDS + Kafka producer
Energy Analytics MS	EnergyAnalytics	Pattern 2: REST + JDBC + RDS + Kafka producer (4 topics)
ArtificialIntelligence MS	(stateless)	AI Pattern: REST + Ollama Server (EC2 t3.large)

> **Note on Flexibility Forecasting (AI):** The ArtificialIntelligence MS is a dedicated microservice (`microservices/ArtificialIntelligence/`) with its own EC2 instance and Terraform configuration (`Quarkus-Terraform/artificialIntelligence/`). It calls Ollama at port 11434 on a separate `t3.large` EC2 host and returns the AI analysis to the caller. It has no RDS and produces no Kafka events directly.

1.3 Information Flow Diagrams

D1 — Asset Registration, Asset Link Activation and Telemetry Ingestion

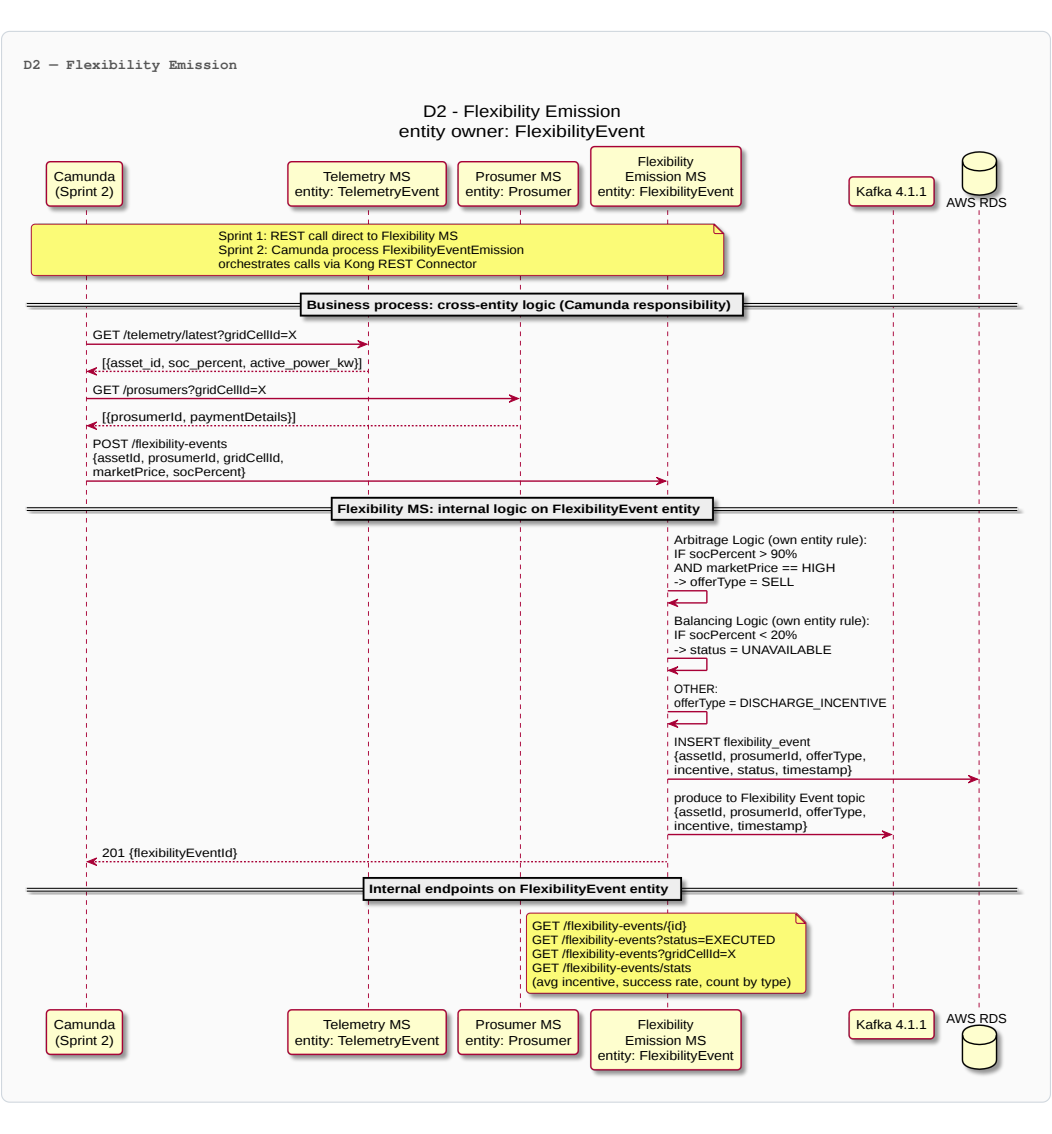
Covers prosumer and operator registration, AssetLink creation, and the continuous telemetry ingestion loop. The Kafka topic lifecycle (create topic on AssetLink POST, delete on DELETE, start/stop `DynamicTopicConsumer` via Telemetry MS) is the Sprint 2 target design. In Sprint 1, topic management and consumer provisioning are done manually.


D1 — Asset Registration, Asset Link Activation and Telemetry Ingestion

D1 - Asset Registration, Asset Link Activation and Telemetry Ingestion
Prosumer
Prosumer MS entity: Prosumer
UtilityOperator MS entity: UtilityOperator + GridCell
AssetLink MS entity: AssetLink
Kafka 4.1.1 (KRaft, EC2:9092)
Telemetry MS entity: TelemetryEvent
AWS RDS
1. Prosumer Registration
POST /prosumers {name, fiscalId, location, paymentDetails}
INSERT prosumer
201 {prosumerId}
Internal logic on Prosumer entity: GET /prosumers/{id} GET /prosumers?gridCellId=X GET /prosumers/{id}/assets
2. UtilityOperator Registration
POST /utilityoperators {name, fiscalId, location}
INSERT utility_operator
INSERT grid_cell {gridCellId, maxCapacityKw, safetyThreshold}
201 {utilityOperatorId}
Internal logic on UtilityOperator entity: GET /gridcells/{id} GET /gridcells?zone=X PUT /gridcells/{id}/threshold
3. Asset Link Activation (Sprint 1: direct | Sprint 2: Camunda orchestrates)
POST /assetlinks {prosumerId, assetId, utilityOperatorId}
INSERT asset_link (status=ACTIVE)
Internal logic on AssetLink entity: GET /assetlinks/{id} GET /assetlinks?prosumerId=X GET /assetlinks?status=ACTIVE DELETE /assetlinks/{id} (deactivate)
AdminClient.createTopics() NewTopic("{assetLinkId}-{utilityOpId}", partitions=1, replication=1)
POST /CreateaTopicConsumer {topicName: "{assetLinkId}-{utilityOpId}"}
new DynamicTopicConsumer(topic, kafka.bootstrap.servers).start()
200 OK
201 {assetLinkId}
4. Telemetry Ingestion (continuous polling loop)
VPPaaS Simulator publishes BESS/Solar/EV JSON events topic: {assetLinkId}-{utilityOpId}
consumer.poll(100ms)
INSERT telemetry_event (asset_id, asset_type, grid_cell_id, soc_percent, active_power_kw, energy_available_kwh, timestamp)
Internal logic on TelemetryEvent entity: GET /telemetry/latest?gridCellId=X GET /telemetry/history?assetId=X GET /telemetry/stats?gridCellId=X (min/max/avg soc_percent, active_power_kw)
5. Asset Link Deactivation (prevents Zombie Assets)
DELETE /assetlinks/{assetLinkId}
UPDATE asset_link (status=INACTIVE)
POST /StopTopicConsumer {topicName: "{assetLinkId}-{utilityOpId}"}
DynamicTopicConsumer.stop()
200 OK
AdminClient.deleteTopics() ("{assetLinkId}-{utilityOpId}")
200 OK

D2 — Flexibility Emission

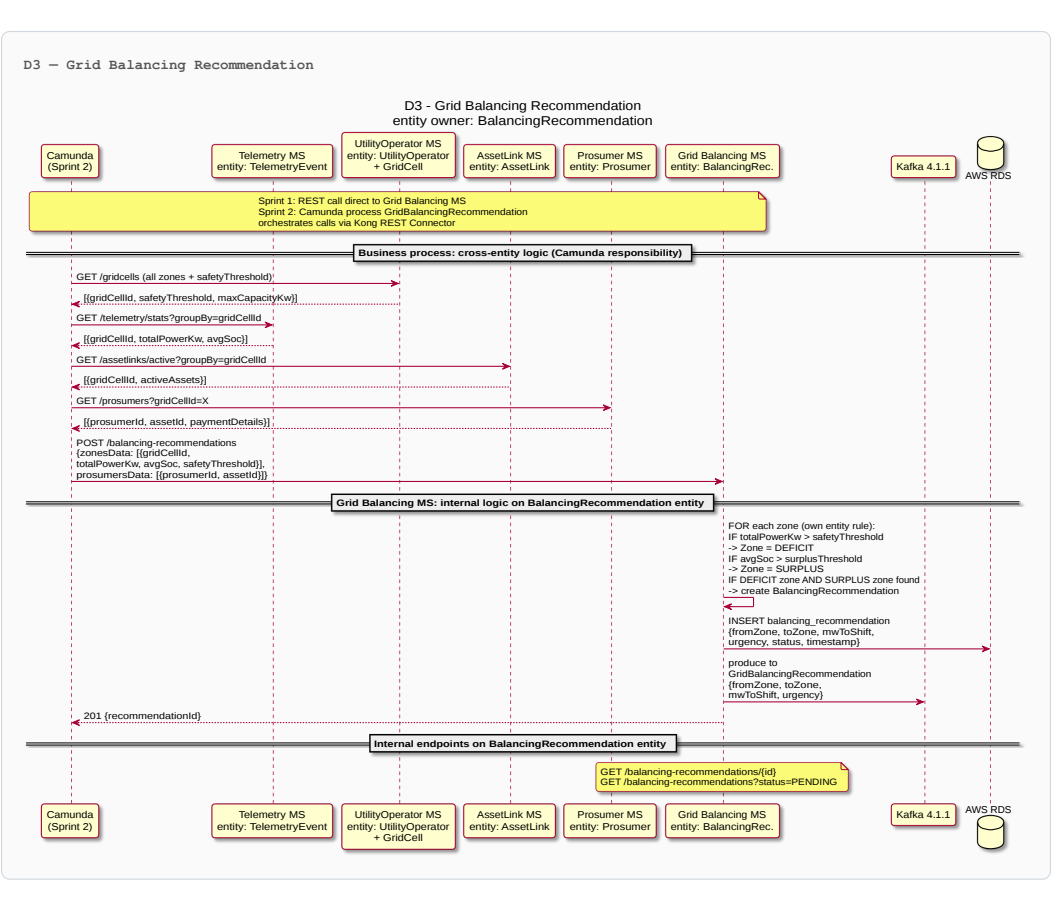
The Flexibility MS is called via REST with the event data. It persists the record and publishes to the `flexibility-offers` Kafka topic. In Sprint 2 the MS will autonomously query the Telemetry MS and Prosumer MS and apply its own business rules.


D2 — Flexibility Emission

D2 - Flexibility Emission
entity owner: FlexibilityEvent
Camunda (Sprint 2)
Telemetry MS entity: TelemetryEvent
Prosumer MS entity: Prosumer
Flexibility Emission MS entity: FlexibilityEvent
Kafka 4.1.1
AWS RDS
Sprint 1: REST call direct to Flexibility MS
Sprint 2: Camunda process FlexibilityEventEmission orchestrates calls via Kong REST Connector
Business process: cross-entity logic (Camunda responsibility)
GET /telemetry/latest?gridCellId=X
[{asset_id, soc_percent, active_power_kw}]
GET /prosumers?gridCellId=X
[{prosumerId, paymentDetails}]
POST /flexibility-events {assetId, prosumerId, gridCellId, marketPrice, socPercent}
Flexibility MS: internal logic on FlexibilityEvent entity
Arbitrage Logic (own entity rule): IF socPercent > 90% AND marketPrice == HIGH -> offerType = SELL
Balancing Logic (own entity rule): IF socPercent < 20% -> status = UNAVAILABLE
OTHER: offerType = DISCHARGE_INCENTIVE
INSERT flexibility_event {assetId, prosumerId, offerType, incentive, status, timestamp}
produce to Flexibility Event topic {assetId, prosumerId, offerType, incentive, timestamp}
201 {flexibilityEventId}
Internal endpoints on FlexibilityEvent entity
GET /flexibility-events/{id} GET /flexibility-events?status=EXECUTED GET /flexibility-events?gridCellId=X GET /flexibility-events/stats (avg incentive, success rate, count by type)

D3 — Grid Balancing Recommendation

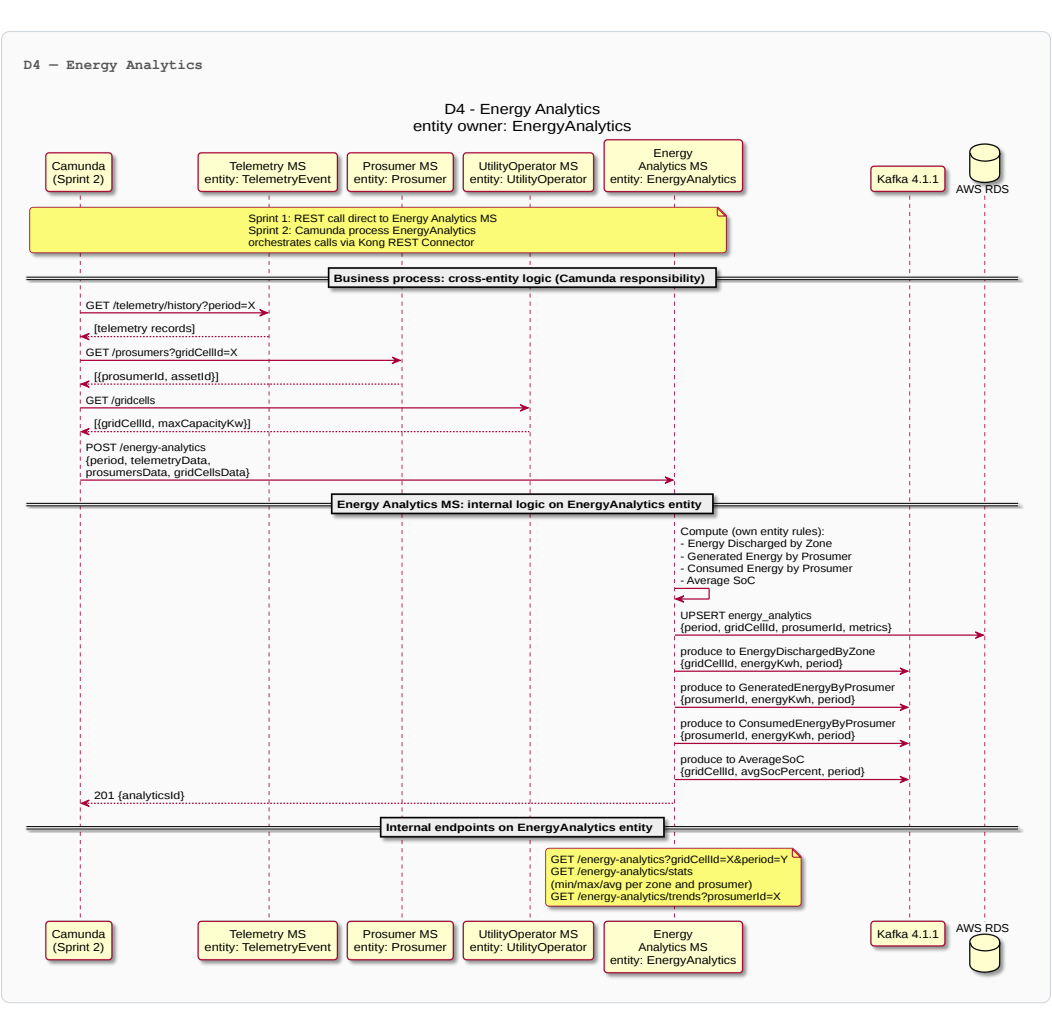
The Grid Balancing MS is called via REST with pre-assembled zone data. It persists and publishes to the `balancing-recommendation` topic. In Sprint 2 it will autonomously query UtilityOperator MS, Telemetry MS, and AssetLink MS.


D3 — Grid Balancing Recommendation

D3 - Grid Balancing Recommendation
entity owner: BalancingRecommendation
Camunda (Sprint 2)
Telemetry MS entity: TelemetryEvent
UtilityOperator MS entity: UtilityOperator + GridCell
AssetLink MS entity: AssetLink
Prosumer MS entity: Prosumer
Grid Balancing MS entity: BalancingRec.
Kafka 4.1.1
AWS RDS
Sprint 1: REST call direct to Grid Balancing MS
Sprint 2: Camunda process GridBalancingRecommendation orchestrates calls via Kong REST Connector
Business process: cross-entity logic (Camunda responsibility)
GET /gridcells (all zones + safetyThreshold)
[{gridCellId, safetyThreshold, maxCapacityKw}]
GET /telemetry/stats?groupBy=gridCell
[{gridCellId, totalPowerKw, avgSoc}]
GET /assetlinks/active?groupBy=gridCell
[{gridCellId, activeAssets}]
GET /prosumers?gridCellId=X
[{prosumerId, assetId, paymentDetails}]
POST /balancing-recommendations {zonesData: [{gridCellId, totalPowerKw, avgSoc, safetyThreshold}], prosumersData: [{prosumerId, assetId}]}
Grid Balancing MS: internal logic on BalancingRecommendation entity
FOR each zone (own entity rule): IF totalPowerKw > safetyThreshold -> Zone = DEFICIT IF avgSoc > surplusThreshold -> Zone = SURPLUS IF DEFICIT zone AND SURPLUS zone found -> create BalancingRecommendation
INSERT balancing_recommendation {fromZone, toZone, mwToShift, urgency, status, timestamp}
produce to GridBalancingRecommendation {fromZone, toZone, mwToShift, urgency}
201 {recommendationId}
Internal endpoints on BalancingRecommendation entity
GET /balancing-recommendations/{id} GET /balancing-recommendations?status=PENDING

D4 — Energy Analytics

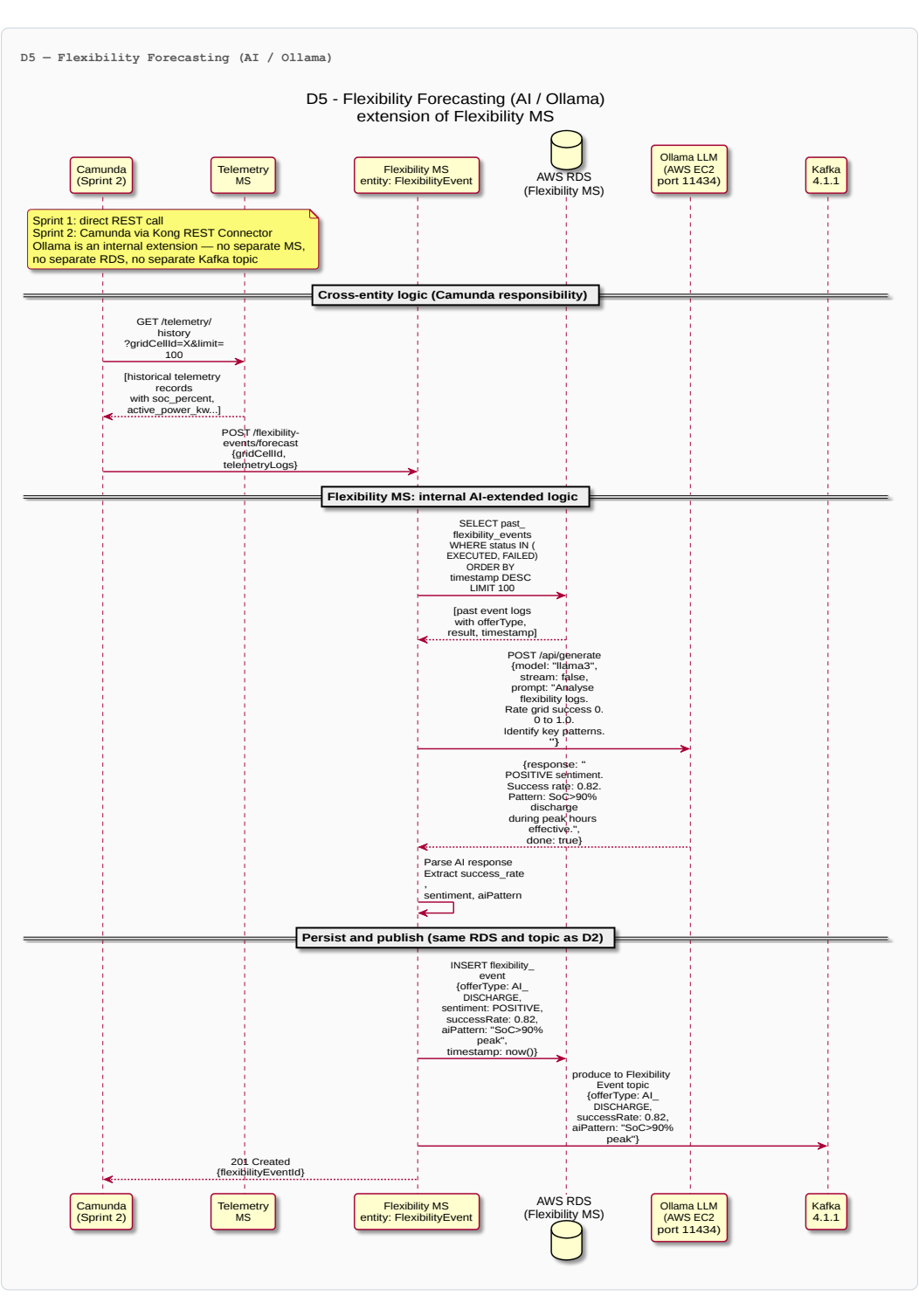
The Energy Analytics MS is called via REST with a pre-computed metric. It persists and routes to one of four Kafka channels (`energy-discharged-zone` , `generated-energy-prosumer` , `consumed-energy-prosumer` , `average-soc`). In Sprint 2 it will autonomously aggregate data from other MS.


D4 — Energy Analytics

D4 - Energy Analytics
entity owner: EnergyAnalytics
Camunda (Sprint 2)
Telemetry MS entity: TelemetryEvent
Prosumer MS entity: Prosumer
UtilityOperator MS entity: UtilityOperator
Energy Analytics MS entity: EnergyAnalytics
Kafka 4.1.1
AWS RDS
Sprint 1: REST call direct to Energy Analytics MS
Sprint 2: Camunda process EnergyAnalytics orchestrates calls via Kong REST Connector
Business process: cross-entity logic (Camunda responsibility)
GET /telemetry/history?period=X
[telemetry records]
GET /prosumers?gridCellId=X
[{prosumerId, assetId}]
GET /gridcells
[{gridCellId, maxCapacityKw}]
POST /energy-analytics {period, telemetryData, prosumersData, gridCellsData}
Energy Analytics MS: internal logic on EnergyAnalytics entity
Compute (own entity rules): - Energy Discharged by Zone - Generated Energy by Prosumer - Consumed Energy by Prosumer - Average SoC
UPSERT energy_analytics {period, gridCellId, prosumerId, metrics}
produce to EnergyDischargedByZone {gridCellId, energyKwh, period}
produce to GeneratedEnergyByProsumer {prosumerId, energyKwh, period}
produce to ConsumedEnergyByProsumer {prosumerId, energyKwh, period}
produce to AverageSoC {gridCellId, avgSocPercent, period}
201 {analyticsId}
Internal endpoints on EnergyAnalytics entity
GET /energy-analytics?gridCellId=X&period=Y GET /energy-analytics/stats (min/max/avg per zone and prosumer) GET /energy-analytics/trends?prosumerId=X

D5 — Flexibility Forecasting (AI / Ollama)

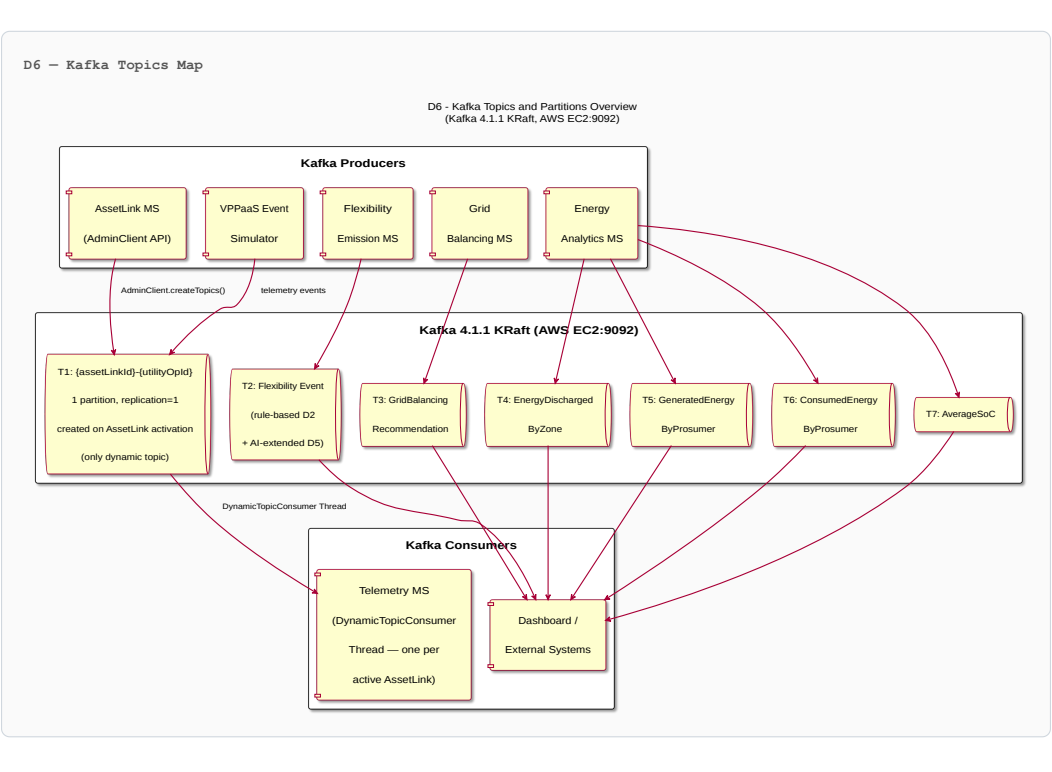
The ArtificialIntelligence MS receives a log text string via REST, forwards it to Ollama (llama3.2), and returns `GRID_STABLE` or `FAILED` . It is a separate, stateless microservice with no database and no Kafka integration.


D5 — Flexibility Forecasting (AI / Ollama)

D5 - Flexibility Forecasting (AI / Ollama)
extension of Flexibility MS
Camunda (Sprint 2)
Telemetry MS
Flexibility MS entity: FlexibilityEvent
AWS RDS (Flexibility MS)
Ollama LLM (AWS EC2 port 11434)
Kafka 4.1.1
Sprint 1: direct REST call
Sprint 2: Camunda via Kong REST Connector
Ollama is an internal extension — no separate MS, no separate RDS, no separate Kafka topic
Cross-entity logic (Camunda responsibility)
GET /telemetry/history?gridCellId=X&limit=100
[historical telemetry records with soc_percent, active_power_kw...]
POST /flexibility-events/forecast {gridCellId, telemetryLogs}
Flexibility MS: internal AI-extended logic
SELECT past_flexibility_events WHERE status IN (EXECUTED, FAILED) ORDER BY timestamp DESC LIMIT 100
[past event logs with offerType, result, timestamp]
POST /api/generate {model: "llama3", stream: false, prompt: "Analyse flexibility logs. Rate grid success 0.0 to 1.0. Identify key patterns."}
{response: " POSITIVE sentiment. Success rate: 0.82. Pattern: SoC>90% discharge during peak hours effective.", done: true}
Parse AI response Extract success_rate , sentiment, aiPattern
Persist and publish (same RDS and topic as D2)
INSERT flexibility_event {offerType: AI_DISCHARGE, sentiment: POSITIVE, successRate: 0.82, aiPattern: "SoC>90% peak", timestamp: now()}
produce to Flexibility Event topic {offerType: AI_DISCHARGE, successRate: 0.82, aiPattern: "SoC>90% peak"}
201 Created {flexibilityEventId}

D6 — Kafka Topics Map

Overview of all Kafka topics, producers, and consumers. The dynamic telemetry topic `{assetLinkId}-{utilityOpId}` is planned for Sprint 2. All other topics are static and created manually at deployment time.


D6 — Kafka Topics Map

D6 - Kafka Topics and Partitions Overview
(Kafka 4.1.1 KRaft, AWS EC2:9092)
Kafka Producers
AssetLink MS (AdminClient API)
VPPaaS Event Simulator
Flexibility Emission MS
Grid Balancing MS
Energy Analytics MS
AdminClient.createTopics()
telemetry events
Kafka 4.1.1 KRaft (AWS EC2:9092)
T1: {assetLinkId}-{utilityOpId} 1 partition, replication=1 created on AssetLink activation (only dynamic topic)
T2: Flexibility Event (rule-based D2 + AI-extended D5)
T3: GridBalancing Recommendation
T4: EnergyDischarged ByZone
T5: GeneratedEnergy ByProsumer
T6: ConsumedEnergy ByProsumer
T7: AverageSoC
DynamicTopicConsumer Thread
Kafka Consumers
Telemetry MS (DynamicTopicConsumer Thread — one per active AssetLink)
Dashboard / External Systems

---

2. Technology Stack and Rationale

2.1 Quarkus 3.27.2 (Java 17)

All eight microservices are built with **Quarkus** rather than Spring Boot. The short version is that Quarkus was built for containerised workloads, and that shows.

Each microservice runs in its own Docker container on EC2, so startup time actually matters; Quarkus boots in milliseconds rather than the several seconds Spring Boot typically takes. More practically, Quarkus has first-class extensions for everything this project needs: Kafka, REST, OpenAPI, and the reactive MySQL client all work out of the box without gluing libraries together. The reactive MySQL client (`io.vertx.mutiny.mysqlclient`) integrates natively with SmallRye Mutiny (`Uni<T>` , `Multi<T>`), which is important for the Telemetry MS; it handles concurrent writes from multiple `DynamicTopicConsumer` threads without blocking the event loop. With Spring Boot and standard JDBC that would require extra configuration to avoid thread starvation under load.

2.2 Apache Kafka 4.1.1 (KRaft mode)

Kafka 4.x dropped ZooKeeper entirely in favour of **KRaft**, and we went with it. The practical reason is simple: ZooKeeper would require a separate EC2 instance just to keep Kafka running, which doubles the infrastructure cost for no real gain in a single-broker setup. KRaft lets the broker manage its own metadata, so one instance does everything. It is also the direction Kafka is going, and there is no point building on a configuration mode that is being phased out.

2.3 AWS RDS MySQL (shared instance)

All microservices share a **single RDS instance** (`db.t4g.micro`) with one schema (`VPPaaS`). The honest reason is cost: `db.t4g.micro` is already the smallest available tier, and running eight of them would be expensive for a coursework project with no real traffic. Sharing one instance also keeps the deployment simpler; there is one RDS endpoint to capture and inject into all `application.properties` files, rather than eight.

In a production system you would want each microservice to own its own database (the database-per-service pattern), but that is a scaling concern that does not apply here.

2.4 Docker Hub as Image Registry

Images are pushed to **Docker Hub** rather than AWS ECR. ECR would mean provisioning another AWS resource, configuring IAM roles for EC2 to pull from it, and generally adding ceremony that is not worth it here. Docker Hub keeps it simple and `docker pull` in the `user_data` script just works, no extra credentials needed on the EC2 side.

2.5 Ollama (llama3.2) for AI Forecasting

The AI Forecasting microservice runs **Ollama** on a dedicated `t3.large` EC2 instance. It exposes a REST API at port 11434 and is reached by the ArtificialIntelligence MS via HTTP. We chose `llama3.2` because it is capable enough for log-analysis prompts without requiring a GPU.

Running the model locally means no per-token API costs and no data leaving the AWS environment. The Quarkus MicroProfile `@RestClient` handles the HTTP call cleanly; there is no third-party LLM SDK involved, just a plain REST call and some JSON parsing.

---

3. Source Code Documentation

3.1 Common Structure

Every microservice follows the same layout:

```
microservices/<ServiceName>/
├── src/main/java/org/acme/
│   ├── <Entity>.java            # Entity class (table schema + reactive CRUD methods)
│   ├── <Entity>Resource.java    # JAX-RS REST resource (HTTP routing)
│   └── KafkaProducer.java       # Kafka emitter (where applicable)
├── src/main/resources/
│   └── application.properties  # Quarkus configuration
├── src/main/docker/
│   ├── Dockerfile.jvm
│   ├── Dockerfile.native
│   └── Dockerfile.native-micro
├── pom.xml
└── mvnw
```

Entity classes use the Quarkus reactive MySQL client (`io.vertx.mutiny.mysqlclient.MySQLPool`) directly; no JPA/Hibernate. JPA adds startup overhead and schema validation that slows down container startup, so the reactive client was used instead: it executes raw SQL and maps rows to POJOs, giving full control over queries and keeping the service lean.

Resource classes are JAX-RS classes annotated with `@Path` . All endpoints return `Uni<Response>` , which integrates with the Vert.x event loop and avoids blocking threads during I/O.

3.2 Prosumer Microservice

Path: `microservices/Prosumer/`

Entities and schema:

```
CREATE TABLE Prosumer (
  id           SERIAL PRIMARY KEY,
  name         TEXT NOT NULL,
  FiscalNumber BIGINT UNSIGNED,
  location     TEXT NOT NULL
);

CREATE TABLE Asset (
  id              SERIAL PRIMARY KEY,
  prosumer_id     BIGINT UNSIGNED,
  asset_id        TEXT NOT NULL,
  asset_type      TEXT NOT NULL,   -- BATTERY | SOLAR | EV_CHARGER
  max_capacity_kw FLOAT
);
```

`Asset` is a child entity of `Prosumer` , linked via `prosumer_id` . Keeping assets in a separate table rather than a JSON column allows standard SQL queries for listing, filtering by type, and joining with telemetry data. The Prosumer MS is responsible for identifying the assets of a prosumer, as defined in the project brief.

On startup the service inserts a seed prosumer (`client1`) if the table is empty, ensuring the database has at least one record for end-to-end testing without manual setup.

REST endpoints:

Method	Path	Description
GET	`/Prosumer`	List all prosumers
GET	`/Prosumer/{id}`	Get prosumer by ID
GET	`/Prosumer?gridCellId={id}`	List prosumers by grid cell
POST	`/Prosumer`	Create prosumer
PUT	`/Prosumer/{id}/{name}/{FiscalNumber}/{location}`	Update prosumer
DELETE	`/Prosumer/{id}`	Delete prosumer
POST	`/Prosumer/{prosumerId}/assets`	Register asset for a prosumer
GET	`/Prosumer/{prosumerId}/assets`	List assets for a prosumer
DELETE	`/Prosumer/{prosumerId}/assets/{assetId}`	Remove an asset

> **Data access pattern:** The Flexibility MS, Grid Balancing MS, and Energy Analytics MS query this service via `GET /Prosumer?gridCellId={id}` to obtain the list of prosumers in a zone before processing their events.

3.3 UtilityOperator Microservice

Path: `microservices/UtilityOperator/`

Entities and schema:

```
CREATE TABLE UtilityOperator (
  id       SERIAL PRIMARY KEY,
  name     TEXT NOT NULL,
  location TEXT NOT NULL
);

CREATE TABLE GridCell (
  id           SERIAL PRIMARY KEY,
  operator_id  BIGINT UNSIGNED,
  grid_cell_id TEXT NOT NULL,
  location     TEXT NOT NULL,
  max_load_mw  FLOAT
);
```

A `GridCell` represents a physical area of the distribution grid and is always managed by exactly one operator. The `max_load_mw` field defines the capacity ceiling of the zone.

On startup, the service seeds a default operator (`ArcoCegoLisbon`) with grid cell `LISBON-DT` (50 MW max load).

> **Planned (Sprint 2):** A `safety_threshold_mw` column will be added to GridCell so the Grid Balancing MS can read per-zone thresholds automatically. Currently that value is passed directly by the caller in the POST body.

REST endpoints:

Method	Path	Description
GET	`/UtilityOperator`	List all operators
GET	`/UtilityOperator/{id}`	Get operator by ID
POST	`/UtilityOperator`	Create operator
PUT	`/UtilityOperator/{id}/{name}/{location}`	Update operator
DELETE	`/UtilityOperator/{id}`	Delete operator
POST	`/UtilityOperator/{operatorId}/grid-cells`	Add a grid cell
GET	`/UtilityOperator/{operatorId}/grid-cells`	List grid cells for an operator
DELETE	`/UtilityOperator/grid-cells/{id}`	Delete a grid cell
PUT	`/UtilityOperator/grid-cells/{id}/{maxLoadMw}`	Update grid cell capacity

3.4 AssetLink Microservice

Path: `microservices/AssetLink/`

Entity and schema:

```
CREATE TABLE AssetLink (
  id                SERIAL PRIMARY KEY,
  idProsumer        BIGINT UNSIGNED,
  idUtilityOperator BIGINT UNSIGNED,
  UNIQUE KEY UC_Loyal (idProsumer, idUtilityOperator)
);
```

The `UNIQUE KEY` on `(idProsumer, idUtilityOperator)` enforces, at the database level, that a prosumer can link to a given operator only once, preventing duplicate topic creation.

The AssetLink MS is a CRUD service: it creates, reads, updates, and deletes asset link records in the database.

The Kafka topic lifecycle management shown in diagram D1 (creating topic `{assetLinkId}-{utilityOperatorId}` via AdminClient and notifying the Telemetry MS via `POST /Telemetry/Consume`) is the intended integration design and is planned for Sprint 2. The Telemetry MS already exposes `POST /Telemetry/Consume` in anticipation of this integration.

REST endpoints:

Method	Path	Description
GET	`/AssetLink`	List all asset links
GET	`/AssetLink/{id}`	Get by ID
GET	`/AssetLink/{idProsumer}/{idUtilityOperator}`	Get by composite key
POST	`/AssetLink`	Create asset link
PUT	`/AssetLink/{id}/{idProsumer}/{idUtilityOperator}`	Update
DELETE	`/AssetLink/{id}`	Delete asset link

> **Planned for Sprint 2:** `GET /AssetLink/active?gridCellId={id}` (list active links by grid cell, needed by Grid Balancing MS) and the Kafka topic lifecycle calls (AdminClient topic creation on POST, topic deletion on DELETE) are not yet implemented.

3.5 Telemetry Ingestion Microservice

Path: `microservices/Telemetry/`

Entity and schema:

```
CREATE TABLE Telemetry (
  id                 SERIAL PRIMARY KEY,
  timeStamp          DATETIME,
  asset_id           BIGINT UNSIGNED,
  asset_type         TEXT NOT NULL,
  grid_cell_id       TEXT NOT NULL,
  State_of_Charge    FLOAT,
  Available_Energy   FLOAT,
  Current_Output     FLOAT,
  Max_Capacity       FLOAT,
  State_of_Health    FLOAT,
  Status             TEXT NOT NULL,
  Current_Generation FLOAT,
  Daily_Total        FLOAT,
  Grid_Voltage       FLOAT,
  Frequency          FLOAT,
  Plug_Status        TEXT NOT NULL,
  Charging_Rate      FLOAT,
  Session_Energy     FLOAT,
  EV_SoC             FLOAT
);
```

A **single wide table** stores all three asset types (BATTERY, SOLAR, EV_CHARGER) with nullable columns for fields that do not apply to every type (e.g. `EV_SoC` is only set for EV charger events). The alternative; a separate table per asset type; would require UNION queries or multiple round-trips when aggregating across assets in a grid cell. The wide-table approach keeps queries simple, at the cost of some null storage.

DynamicTopicConsumer (`DynamicTopicConsumer.java`) is a plain Java `Thread` subclass. Each instance:

1. Creates its own `KafkaConsumer` connected to the bootstrap server.
2. Subscribes to a single named topic.
3. Runs a `consumer.poll(100ms)` loop until `stop()` is called.
4. Parses each record's `asset_type` field and writes the appropriate columns.

Using a plain `Thread` rather than a Quarkus/Vert.x managed executor was deliberate: the consumer lifecycle (start/stop per AssetLink) is driven by external REST calls from the AssetLink MS, not by the framework. A plain thread can be held in a `ConcurrentHashMap<String, DynamicTopicConsumer>` keyed by topic name and stopped on demand, without framework interference.

> **Data access pattern:** The Flexibility MS, Grid Balancing MS, Energy Analytics MS, and the AI extension within the Flexibility MS all query the Telemetry MS via REST (`GET /Telemetry/grid/{gridCellId}` , `GET /Telemetry/asset/{assetId}`) to obtain the latest and historical telemetry readings. The Telemetry MS does **not** publish to any Kafka topic; it exposes data exclusively via REST.

REST endpoints (provisioning):

Method	Path	Description
POST	`/Telemetry/Consume`	Start a DynamicTopicConsumer for a topic

> **Planned for Sprint 2:** `DELETE /Telemetry/Consume/{topicName}` to stop and remove a running consumer when an AssetLink is deactivated. The current implementation starts consumers but does not yet expose an endpoint to stop them.

REST endpoints (data):

Method	Path	Description
GET	`/Telemetry`	List all records
GET	`/Telemetry/{id}`	Get by ID
GET	`/Telemetry/asset/{assetId}`	Records for an asset (by timestamp DESC)
GET	`/Telemetry/grid/{gridCellId}`	Records for a grid cell (by timestamp DESC)

3.6 Flexibility Emission Microservice

Path: `microservices/FlexibilityEvent/`

This microservice owns the `FlexibilityEvent` entity. It receives a pre-evaluated event via REST, persists it to RDS, publishes it to Kafka, and marks it `PUBLISHED` .

Entity and schema:

```
CREATE TABLE FlexibilityEvent (
  id                     SERIAL PRIMARY KEY,
  timestamp              DATETIME NOT NULL,
  asset_id               TEXT NOT NULL,
  prosumer_id            BIGINT,
  grid_cell_id           TEXT NOT NULL,
  logic_type             TEXT NOT NULL,
  proposed_action        TEXT NOT NULL,
  incentive_value        FLOAT,
  target_value_kw        FLOAT,
  telemetry_reference_id BIGINT,
  status                 TEXT NOT NULL   -- PENDING | PUBLISHED | EXECUTED | FAILED | UNAVAILABLE
);
```

The caller is responsible for setting `logic_type` (e.g. `ARBITRAGE` , `SAFETY` , `BALANCING`) and `proposed_action` (e.g. `SELL` , `DISCHARGE_INCENTIVE`) in the POST body. The MS does not evaluate these values itself.

> **Planned (Sprint 2):** The arbitrage/safety/balancing business rules will be implemented inside the MS itself, so the caller only needs to pass raw telemetry values and the MS will decide the `logic_type` and `proposed_action` autonomously. Currently the caller is responsible for that evaluation.

Kafka producer: `@Channel("flexibility-offers")` → topic `flexibility-offers` (Quarkus default: channel name = topic name when no explicit mapping is configured). Published payload:

```
{
  "event_id": 42,
  "action": "SELL",
  "asset_id": "560987123",
  "prosumer_id": 5,
  "value_kw": 7.2
}
```

REST endpoints:

Method	Path	Description
POST	`/Flexibility/Emit`	Evaluate telemetry and emit a flexibility event
GET	`/Flexibility`	List all events
GET	`/Flexibility/{id}`	Get event by ID
GET	`/Flexibility/status/{status}`	Filter by status
GET	`/Flexibility/grid/{gridCellId}`	Filter by grid cell

3.7 Grid Balancing Microservice

Path: `microservices/GridBalancing/`

Entity and schema:

```
CREATE TABLE GridBalancing (
  id                 SERIAL PRIMARY KEY,
  timestamp          DATETIME NOT NULL,
  source_grid_cell   TEXT NOT NULL,
  target_grid_cell   TEXT NOT NULL,
  recommended_action TEXT NOT NULL,
  power_kw           FLOAT,
  status             TEXT NOT NULL   -- PENDING | PUBLISHED
);
```

The caller supplies `source_grid_cell` , `target_grid_cell` , `recommended_action` , and `power_kw` in the POST body. The MS persists the record, publishes it to Kafka, and marks it `PUBLISHED` .

> **Planned (Sprint 2):** The MS will autonomously query UtilityOperator MS, Telemetry MS, and AssetLink MS via REST to aggregate zone data, classify zones as DEFICIT or SURPLUS, and generate the recommendation itself. The intended logic is: `FOR each zone: IF totalPowerKw > safetyThreshold → DEFICIT IF avgSoc > surplusThreshold → SURPLUS IF DEFICIT zone and SURPLUS zone found → create BalancingRecommendation` Currently this cross-entity aggregation and classification is the caller's responsibility.

Kafka producer: `@Channel("balancing-recommendation")` → topic `balancing-recommendation` (Quarkus default: channel name = topic name). Published payload:

```
{
  "event_id": 1,
  "recommended_action": "TRANSFER",
  "source_grid_cell": "LISBON-WEST",
  "target_grid_cell": "LISBON-DT",
  "power_kw": 5.0
}
```

REST endpoints:

Method	Path	Description
POST	`/GridBalancing/Emit`	Create a balancing recommendation
GET	`/GridBalancing`	List all recommendations
GET	`/GridBalancing/{id}`	Get by ID
GET	`/GridBalancing/status/{status}`	Filter by status
GET	`/GridBalancing/source/{sourceGridCell}`	Filter by source grid cell
GET	`/GridBalancing/target/{targetGridCell}`	Filter by target grid cell

3.8 Energy Analytics Microservice

Path: `microservices/EnergyAnalytics/`

Entity and schema:

```
CREATE TABLE EnergyAnalytics (
  id           SERIAL PRIMARY KEY,
  timestamp    DATETIME NOT NULL,
  metric_type  TEXT NOT NULL,
  reference_id TEXT NOT NULL,
  metric_value FLOAT NOT NULL,
  unit         TEXT NOT NULL,
  status       TEXT NOT NULL
);
```

`metric_type` is one of: `ENERGY_DISCHARGED_BY_ZONE` , `GENERATED_ENERGY_BY_PROSUMER` , `CONSUMED_ENERGY_BY_PROSUMER` , `AVERAGE_SOC` . A generic row structure was chosen over four separate tables because the analytics schema is identical for all metrics; only `metric_type` and `reference_id` (gridCellId or prosumerId) differ.

The caller supplies the pre-computed `metric_type` , `reference_id` , `metric_value` , and `unit` in the POST body. The MS persists the record and routes it to the correct Kafka channel based on `metric_type` .

> **Planned (Sprint 2):** The MS will autonomously query Telemetry MS, Prosumer MS, and UtilityOperator MS to compute metrics for a requested time window. Currently the caller is responsible for providing the computed values.

Four Kafka producer channels, one per metric type:

Channel	Kafka topic (default = channel name)
`energy-discharged-zone`	`energy-discharged-zone`
`generated-energy-prosumer`	`generated-energy-prosumer`
`consumed-energy-prosumer`	`consumed-energy-prosumer`
`average-soc`	`average-soc`

REST endpoints:

Method	Path	Description
POST	`/EnergyAnalytics/Publish`	Record and publish a metric
GET	`/EnergyAnalytics`	List all metrics
GET	`/EnergyAnalytics/{id}`	Get metric by ID
GET	`/EnergyAnalytics/metric_type/{metricType}`	Filter by metric type

3.9 ArtificialIntelligence Microservice

Path: `microservices/ArtificialIntelligence/`

This microservice is stateless: it has no database and produces no Kafka events. It acts as a REST proxy between the caller and the Ollama LLM daemon running on the same EC2 host at port 11434.

Key classes:

Class	Role
`ForecastingResource.java`	`POST /Forecasting/Analyze` — receives a `ForecastRequest` DTO
`OllamaRestClient.java`	MicroProfile `@RegisterRestClient` targeting Ollama's `/api/generate`
`ForecastRequest.java`	Input DTO carrying the `log_text` string

Request to Ollama:

```
{
  "model": "llama3.2",
  "stream": false,
  "prompt": "Analyse flexibility logs. Rate grid success 0.0 to 1.0. Identify key patterns.\n\n<log_text>"
}
```

`stream: false` was chosen so the full response arrives in a single HTTP reply, simplifying response parsing.

REST endpoint:

Method	Path	Description
POST	`/Forecasting/Analyze`	Submit log text for AI analysis

---

4. Terraform Scripts Documentation

4.1 RDS; Shared MySQL Database

File: `RDS-Terraform/RDSCreation-S3statefile.tf`

```
resource "aws_db_instance" "vppaaas_db" {
  identifier          = "vppaas-db"
  engine              = "mysql"
  engine_version      = "8.0"
  instance_class      = "db.t4g.micro"
  allocated_storage   = 20
  db_name             = "VPPaaS"
  username            = "teste"
  password            = "testeteste"
  publicly_accessible    = true
  skip_final_snapshot    = true
  vpc_security_group_ids = [aws_security_group.rds_sg.id]
}
```

Choices: - `db.t4g.micro` : ARM-based Graviton instance, cheaper than the equivalent x86 tier and sufficient for demo workloads. - `publicly_accessible = true` : microservices run on separate EC2 instances and need to reach the database. In production this would be `false` , with microservices inside a private subnet. - `skip_final_snapshot = true` : suitable for a coursework environment where teardown should be clean and fast. - **Remote state in S3**: the Terraform state file is stored in an S3 bucket so that subsequent Terraform runs can read the RDS endpoint via `terraform_remote_state` .

4.2 Kafka; EC2 KRaft Broker

File: `Kafka/EC2ChangeKafkaConfiguration.tf`

```
resource "aws_instance" "kafka_instance" {
  ami           = "ami-07ff62358b87c7116"
  instance_type = "t3.small"
  key_name      = "vockey"
  vpc_security_group_ids = [aws_security_group.kafka_sg.id]
  user_data     = file("creation.sh")
  tags = { Name = "Kafka-KRaft" }
}
```

`Kafka/creation.sh` runs on first boot and: 1. Installs Java 17. 2. Downloads Kafka 4.1.1 from the Apache mirror. 3. Generates a KRaft UUID with `kafka-storage.sh random-uuid` . 4. Formats storage with that UUID. 5. Patches `server.properties` to set `advertised.listeners` to the instance's public DNS. 6. Starts the broker with `kafka-server-start.sh` .

Key `server.properties` settings:

```
process.roles=broker,controller
node.id=1
controller.quorum.voters=1@localhost:9093
listeners=PLAINTEXT://0.0.0.0:9092,CONTROLLER://0.0.0.0:9093
advertised.listeners=PLAINTEXT://<EC2_PUBLIC_DNS>:9092
```

`process.roles=broker,controller` runs both roles on the same node, which is the standard single-node KRaft configuration.

4.3 Microservice EC2 Instances (per-service template)

Files: `Quarkus-Terraform/<service>/EC2InstallQuarkus.tf` + `quarkus.sh`

Seven of the eight microservice deployments use the same Terraform template (the exception is the ArtificialIntelligence MS, described in §4.4):

```
resource "aws_instance" "quarkus_instance" {
  ami           = "ami-0eb38b817b93460ac"
  instance_type = "t3.small"
  key_name      = "vockey"
  vpc_security_group_ids = [aws_security_group.quarkus_sg.id]
  user_data     = file("quarkus.sh")
}
```

`quarkus.sh` (user data script):

```
#!/bin/bash
docker login -u <DOCKERHUB_USERNAME> -p <DOCKERHUB_PASSWORD>
docker pull <DOCKERHUB_USERNAME>/<service-name>:latest
docker run -d -p 8080:8080 <DOCKERHUB_USERNAME>/<service-name>:latest
```

Flexibility MS; additional Ollama setup: The ArtificialIntelligence MS has its own Terraform configuration at `Quarkus-Terraform/artificialIntelligence/EC2OllamaConfiguration.tf` . It provisions a `t3.large` EC2 instance with 50 GB of disk, opening ports 22, 8080, and 11434. The `creation.sh` script installs Ollama and pulls the `llama3.2` model on first boot.

---

5. Installation Procedures

5.1 Prerequisites

Requirement	Details
AWS credentials	`AWS_ACCESS_KEY_ID` , `AWS_SECRET_ACCESS_KEY` , `AWS_SESSION_TOKEN`
AWS region	`us-east-1`
Docker Hub account	Username and password/access token
Terraform ≥ 1.0	In `$PATH`
Java 17 + Maven	To build Quarkus projects locally
Docker	To build and push images

5.2 Configure Credentials

Edit `access.sh` with real values:

```
export AWS_ACCESS_KEY_ID=<your-key>
export AWS_SECRET_ACCESS_KEY=<your-secret>
export AWS_SESSION_TOKEN=<your-token>
export AWS_DEFAULT_REGION=us-east-1

export DOCKERHUB_USERNAME=<your-dockerhub-username>
export DOCKERHUB_PASSWORD=<your-dockerhub-password>

export JAVA_HOME=<path-to-jdk17>
```

> **Important:** `access.sh` must never be committed to version control. It is listed in `.gitignore` .

```
source access.sh
```

5.3 Full Automated Deployment

```
source access.sh
./DeploymentAutomation-ubuntu.sh   # Linux/Ubuntu
# or
./DeploymentAutomation-macOS.sh    # macOS
```

The script executes the following steps in order:

1. **RDS provisioning**: `terraform init && terraform apply` in `RDS-Terraform/` . The RDS endpoint is captured from Terraform output.
2. **Kafka provisioning**: EC2 instance for Kafka KRaft. The public DNS is captured.
3. **Microservice build and deploy** — for each of the 8 services:
4. `application.properties` is updated with the RDS endpoint and Kafka bootstrap server.
5. `./mvnw clean package -Dquarkus.container-image.push=true` builds and pushes the Docker image.
6. `quarkus.sh` in the Terraform directory is updated with Docker Hub credentials.
7. `terraform apply` provisions the EC2 instance, which pulls and starts the container.
8. For the ArtificialIntelligence MS: Ollama is also installed on the same instance and `llama3.2` is pulled.

5.4 Create Static Kafka Topics

After the Kafka EC2 instance is running, create the static topics (T2–T7) once:

```
ssh -i vockey.pem ubuntu@<KAFKA_EC2_PUBLIC_DNS>

cd /opt/kafka

bin/kafka-topics.sh --bootstrap-server localhost:9092 \
  --create --topic "flexibility-offers"          --partitions 1 --replication-factor 1
bin/kafka-topics.sh --bootstrap-server localhost:9092 \
  --create --topic "balancing-recommendation"    --partitions 1 --replication-factor 1
bin/kafka-topics.sh --bootstrap-server localhost:9092 \
  --create --topic "energy-discharged-zone"      --partitions 1 --replication-factor 1
bin/kafka-topics.sh --bootstrap-server localhost:9092 \
  --create --topic "generated-energy-prosumer"   --partitions 1 --replication-factor 1
bin/kafka-topics.sh --bootstrap-server localhost:9092 \
  --create --topic "consumed-energy-prosumer"    --partitions 1 --replication-factor 1
bin/kafka-topics.sh --bootstrap-server localhost:9092 \
  --create --topic "average-soc"                 --partitions 1 --replication-factor 1

bin/kafka-topics.sh --bootstrap-server localhost:9092 --list
```

The dynamic topic `{assetLinkId}-{utilityOperatorId}` (T1) is created automatically by the AssetLink MS on each POST and deleted on each DELETE; no manual step required.

5.5 Run the VPPaaS EventProducer (Simulator)

```
java -jar VPPaaS-EventProducer/VPPaaSSimulator.jar \
  --broker-list <KAFKA_EC2_PUBLIC_DNS>:9092 \
  --throughput 10 \
  --filterprefix <assetLinkId>
```

Flag	Description
`--broker-list`	Kafka bootstrap address
`--throughput`	Messages per second per topic
`--filterprefix`	Only publish to topics whose name starts with this value (the `assetLinkId`)

The simulator discovers active topics by querying Kafka metadata and filtering by prefix, so it begins publishing as soon as the AssetLink MS creates the topic.

5.6 Teardown

```
./UndeploymentAutomation-all.sh
```

Runs `terraform destroy -auto-approve` in reverse order: all 8 microservice EC2 instances → Kafka → RDS.

---

6. Parametrizations Reference

6.1 `application.properties` ; Standard Template

All services that connect to both MySQL and Kafka use this template:

```
# Database
quarkus.datasource.db-kind=mysql
quarkus.datasource.username=teste
quarkus.datasource.password=testeteste
%prod.quarkus.datasource.reactive.url=mysql://<RDS_ENDPOINT>:3306/VPPaaS

# Kafka (injected by deployment script for services that produce/consume)
kafka.bootstrap.servers=<KAFKA_EC2_PUBLIC_DNS>:9092

# Docker image publication
quarkus.container-image.build=true
quarkus.container-image.push=true
quarkus.container-image.group=<DOCKERHUB_USERNAME>
quarkus.container-image.name=<service-name>
quarkus.container-image.tag=latest

# Swagger UI
quarkus.swagger-ui.path=swagger-ui
quarkus.swagger-ui.always-include=true
```

The `%prod.` prefix means the `datasource.reactive.url` is only applied in the `prod` profile (running on EC2). In `dev` profile, Quarkus Dev Services spins up a local MySQL container automatically.

ArtificialIntelligence MS differs — it has no database block, and adds:

```
%prod.quarkus.rest-client.ollama-api.url=http://172.17.0.1:11434
```

`172.17.0.1` is the Docker bridge gateway IP, which allows the containerised AI MS to reach the Ollama daemon running on the EC2 host. In `dev` profile, the URL defaults to `localhost:11434` .

6.2 `access.sh` ; Environment Variables

Variable	Where used	Description
`AWS_ACCESS_KEY_ID`	Terraform + AWS CLI	IAM access key
`AWS_SECRET_ACCESS_KEY`	Terraform + AWS CLI	IAM secret
`AWS_SESSION_TOKEN`	Terraform + AWS CLI	Session token (AWS Academy temporary credentials)
`AWS_DEFAULT_REGION`	Terraform	Deployment region (`us-east-1`)
`DOCKERHUB_USERNAME`	Deployment script	Docker Hub account for image push/pull
`DOCKERHUB_PASSWORD`	Deployment script	Docker Hub password or access token
`JAVA_HOME`	Maven build	Path to JDK 17

6.3 Variables Injected by the Deployment Script

Variable	Source	Injected into
`RDS_ENDPOINT`	`terraform output` from `RDS-Terraform/`	All `application.properties` (`%prod.quarkus.datasource.reactive.url`)
`KAFKA_BOOTSTRAP_SERVER`	`terraform output` from `Kafka/`	`application.properties` of Telemetry, FlexibilityEvent, GridBalancing, EnergyAnalytics, AssetLink
`DOCKERHUB_USERNAME`	`access.sh`	`quarkus.container-image.group` in `application.properties` ; credentials in `quarkus.sh`

6.4 Kafka Channel-to-Topic Mapping

None of the `application.properties` files contain explicit `mp.messaging.outgoing.*` topic mappings. When no `topic` property is configured, Quarkus SmallRye Reactive Messaging uses the **channel name as the Kafka topic name** by default. The `kafka.bootstrap.servers` property is injected at deploy time by the deployment script.

Service	Channel name in code	Kafka topic (= channel name)
FlexibilityEvent	`flexibility-offers`	`flexibility-offers`
GridBalancing	`balancing-recommendation`	`balancing-recommendation`
EnergyAnalytics	`energy-discharged-zone`	`energy-discharged-zone`
EnergyAnalytics	`generated-energy-prosumer`	`generated-energy-prosumer`
EnergyAnalytics	`consumed-energy-prosumer`	`consumed-energy-prosumer`
EnergyAnalytics	`average-soc`	`average-soc`

6.5 AWS Infrastructure Summary

```
AWS us-east-1
│
├── RDS MySQL db.t4g.micro — schema VPPaaS
│   └── Tables: Prosumer, Asset, UtilityOperator, GridCell, AssetLink,
│               Telemetry, FlexibilityEvent, GridBalancing, EnergyAnalytics
│
├── EC2 t3.small  — Kafka 4.1.1 KRaft          (port 9092)
├── EC2 t3.large  — Camunda 8.8.9              (ports 8080, 8081, 8082)
├── EC2 t3.small  — Kong API Gateway           (ports 8000, 8001)
├── EC2 t3.small  — Konga Admin UI             (port 1337)
│
├── EC2 t3.small  — Prosumer MS               (port 8080)
├── EC2 t3.small  — UtilityOperator MS        (port 8080)
├── EC2 t3.small  — AssetLink MS              (port 8080)
├── EC2 t3.small  — Telemetry MS              (port 8080)
├── EC2 t3.small  — FlexibilityEvent MS       (port 8080)
├── EC2 t3.small  — GridBalancing MS          (port 8080)
├── EC2 t3.small  — EnergyAnalytics MS        (port 8080)
└── EC2 t3.large  — ArtificialIntelligence MS (port 8080)
                  └── Ollama llama3.2 on host (port 11434)
```

All EC2 security groups open application ports to `0.0.0.0/0` . This is appropriate for a coursework/demonstration environment. A production deployment would restrict ingress to VPC-internal CIDRs and place microservices behind a load balancer.